

Lecture 2: Writing Our First Spring Boot Application

1. What Are We Doing Today?

In this lecture, we are going to write our first Spring Boot application.

The goal is not to understand every internal concept immediately. That is intentional.

Spring Boot hides a lot of complexity in the beginning so that developers can quickly start building applications. In this lecture, we will experience how fast Spring Boot helps us create a working web application, and in later lectures, we will slowly understand what is happening behind the scenes.

2. What Are We Building?

We are going to build a simple web application where the browser sends a request to our Spring Boot application, and the application sends back a response.

The flow will look like this:

```
Browser
  |
  | Request
  v
Spring Boot Application
  |
  | Response
  v
Browser
```

We will create one endpoint:

```
localhost:8080/hello
```

When we open this URL in the browser, it will return:

```
Hello World
```

At first, this looks very simple.

But many important questions are hidden behind this simple

```
How does the browser talk to Java?  
How does Java understand URLs?  
Who starts the server?  
Who listens on port 8080?  
How does /hello get connected to our Java method?
```

We will not answer all of these deeply today, but we will create curiosity for the upcoming lectures.

3. What is a Port?

Before running our Spring Boot application, we need to understand the idea of a **port**.

Suppose your laptop is connected to a network.

Your laptop has an IP address.

Example:

```
192.168.1.10
```

Now imagine many applications are running on the same laptop:

```
Chrome  
WhatsApp  
Zoom  
Spotify  
Spring Boot application
```

If some data arrives at your laptop, how will the operating system know which application should receive that data?

This is where **port numbers** come in.

IP Address vs Port Number

An **IP address** identifies the machine.

Example:

```
192.168.1.10
```

A **port number** identifies the application running inside that machine.

Example:

```
192.168.1.10:8080
```

Meaning:

```
Machine      = 192.168.1.10
Application  = Listening on port 8080
```

So the IP address brings the data to the correct machine, and the port number helps the operating system deliver that data to the correct application.

Visual Example

```
Internet
|
v

192.168.1.10
-----
| Chrome           Port 5000 |
| WhatsApp         Port 5222 |
| Zoom             Port 8801 |
| Spring Boot App  Port 8080 |
-----
```

The IP address tells us **which machine** to reach.

The port number tells us **which application** on that machine request.

4. What is Localhost?

Students often get confused between `localhost`, IP address, and port number.

In simple words:

```
localhost = your own machine
```

`localhost` is equivalent to:

```
127.0.0.1
```

So when we write:

```
localhost:8080
```

it means:

```
Send the request to my own machine on port 8080.
```

If our Spring Boot application is running on port `8080`, then the browser can send a request to it using:

```
http://localhost:8080
```

5. How Does the Browser Know to Use?

When we type a normal website URL like:

```
www. .in
```

we usually do not mention any port number.

That is because browsers use default ports.

For example:

Protocol	Default Port
http	80
https	443

So when we write:

```
https://www. .in
```

the browser automatically assumes port 443.

When we write:

```
http://www. .in
```

the browser automatically assumes port 80.

But when we are running a local Spring Boot application, it usually runs on port 8080, so we mention it manually:

```
http://localhost:8080
```

Important point:

DNS gives the IP address of a domain.

The port is either mentioned in the URL or decided by the

For example:

```
https://www. .in
```

Here:

```
DNS resolves www. .in to an IP address.  
HTTPS tells the browser to use port 443 by default.
```

6. Development Setup

For this series, we will use **IntelliJ IDEA**.

You can use any editor, but IntelliJ is widely preferred for Java and Spring Boot development because it has strong support for:

```
Java  
Maven  
Spring Boot  
Code navigation  
Auto-completion  
Project structure  
Debugging
```

Before creating the project, make sure Java is installed and working on your system.

You can verify Java installation using:

```
java -version
```

If Java is not installed, install it first before moving ahead.

7. Why Does Spring Initializr Exist

Imagine starting a Java web project from scratch.

Before writing even a single line of business logic, we would setup tasks:

```
Create folder structure
Create Maven configuration
Select Spring Boot version
Select Java version
Add required dependencies
Configure package structure
Prepare project metadata
```

This setup work is repetitive.

Every Spring Boot project needs a similar basic structure.

To solve this problem, the Spring team created **Spring Initializr**.

What is Spring Initializr?

Spring Initializr is a project generator for Spring applications.

It creates the basic project skeleton for us so that we can quickly start writing application logic.

In simple words:

| Spring Initializr helps us create a ready-to-use Spring Boot project.

It allows us to choose:

```
Project type
Programming language
Spring Boot version
Project metadata
Packaging type
Java version
Dependencies
```

8. What is a Dependency?

When we build a Java project, we often need external libraries.

For example, if we want our Java application to connect with MySQL connector library.

That reusable external code is called a **library**.

When our project needs that library to work, it becomes a **dependency**.

Simple definition:

| A dependency is an external library required by our project.

Examples:

```
Spring Web  
MySQL Connector  
Spring Data JPA  
Spring Security  
Lombok  
Hibernate
```

In Maven projects, dependencies are added inside the `pom.xml` file.

9. Choosing the Right Spring Boot Version

On Spring Initializr, we may see versions like:

```
Stable release  
SNAPSHOT  
RC
```

SNAPSHOT

A `SNAPSHOT` version is a work-in-progress version.

It may contain bugs and is not considered final.

RC

RC means **Release Candidate**.

It is almost ready but still not the final stable release.

Stable Release

For beginners and real projects, we should prefer stable release.

A stable release is tested and suitable for normal development.

10. JAR Packaging in Spring Boot

Modern Spring Boot applications usually use **JAR** packaging.

JAR stands for:

Java Archive

A Spring Boot JAR can contain our application code along with the required setup to run the application.

This is one reason Spring Boot applications are easy to run.

Instead of manually deploying to an external server, we can run the application directly.

11. Project Tour

After creating the project from Spring Initializr and opening it in IntelliJ, we will see a standard Spring Boot project structure.

Common important files and folders include:

```
src/main/java  
src/main/resources  
pom.xml
```

```
application.properties  
Main application class
```

src/main/java

This folder contains our Java source code.

src/main/resources

This folder contains configuration files and static resources.

pom.xml

This is the Maven configuration file.

It contains project information and dependencies.

application.properties

This file is used to configure our Spring Boot application.

For example, we can change the server port from here.

Main Application Class

This is the class from where the Spring Boot application starts.

It usually contains the `main()` method.

Example:

```
@SpringBootApplication  
public class FirstSpringBootApplication {  
  
    public static void main(String[] args) {  
        SpringApplication.run(FirstSpringBootApplication.class,  
            args);  
    }  
}
```

For now, we only need to understand that this class starts our application.

We will understand `@SpringBootApplication` deeply later.

12. What is a Controller?

A **Controller** is the entry point for incoming web requests.

Simple explanation:

| A controller is like the receptionist of our application.

When a request comes to the application, the controller receives it and decides what response should be sent back.

For example:

```
Browser sends request to /hello
Controller receives the request
Controller returns Hello World
```

13. What is `@RestController` ?

`@RestController` is an annotation in Spring.

It tells Spring:

```
This class can handle HTTP requests and return data directly as a response.
```

Example:

```
@RestController
public class HelloController {

}
```

This annotation makes the class eligible to receive web requests.
We will understand annotations and controllers in detail later.

14. Creating Our First Endpoint

Now we can create a simple controller.

Example:

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello World";
    }
}
```

Here:

```
@RestController
```

tells Spring that this class can handle HTTP requests.

```
@GetMapping("/hello")
```

tells Spring that this method should run when a GET request comes to `/hello`.

```
sayHello()
```

is the Java method that gets executed.

```
return "Hello World";
```

is the response sent back to the browser.

So when we open:

```
http://localhost:8080/hello
```

the browser displays:

```
Hello World
```

15. What Happened When We Ran the Application?

When we run the Spring Boot application, we may see a message like:

```
Tomcat started on port 8080
```

This is very important.

We did not install Tomcat separately.

We did not write socket programming code.

We did not manually create a server.

Still, Tomcat started.

Why?

Because Spring Boot includes and configures an embedded Tomcat server for us when we use the web dependency.

Spring Boot starts the application and also starts the embedded Tomcat server.

Tomcat is the actual server listening on the port.

So the correct understanding is:

```
Spring Boot starts and configures the application.  
Embedded Tomcat listens for HTTP requests.  
Spring MVC maps the request to the correct controller method.
```

We will understand this complete flow properly in upcoming I

16. Changing the Port Number

By default, Spring Boot runs on port:

```
8080
```

We can change the port using the `application.properties` file.

Example:

```
server.port=9090
```

After changing this and restarting the application, the app will run on:

```
http://localhost:9090
```

Now our endpoint will be:

```
http://localhost:9090/hello
```

Important point:

Spring Boot reads the configuration, but the embedded Tomcat server listens on the configured port.

So if we set:

```
server.port=9090
```

Tomcat starts listening on port `9090`.

17. Complete Flow of Our First S Application

The complete flow looks like this:

```
Browser sends request to localhost:8080/hello
|
v
Request reaches embedded Tomcat
|
v
Spring MVC checks the available mappings
|
v
/hello is matched with sayHello()
|
v
sayHello() returns "Hello World"
|
v
Response goes back to browser
|
v
Browser displays Hello World
```

This is what happens behind one simple line in the browser:

```
localhost:8080/hello
```

18. Why This Feels So Fast

In less than a few minutes, we created a working web application.

We wrote very little code:

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public String sayHello() {
        return "Hello World";
    }
}
```

But behind this simple code, Spring Boot did many things for us:

```
Created application context
Started embedded Tomcat
Configured Spring MVC
Detected controller class
Registered /hello endpoint
Handled HTTP request and response
Returned data to browser
```

This is the power of Spring Boot.

It allows developers to focus more on business logic instead of repeating infrastructure setup again and again.

19. Then Why Learn Internals?

A common question is:

If Spring Boot does everything automatically, why should we learn Tomcat, Servlets, Maven, Spring Core, MVC, IoC, and Dependency Injection?

The answer is simple:

Spring Boot helps us start fast, but internals help us solve real problems.

Writing basic annotations is easy.

For example:

```
@RestController  
@GetMapping("/hello")
```

Many tools can generate this code today.

But real engineering starts when something does not work.

20. Real Problems Require Internal Understanding

Suppose we create an endpoint, but the browser shows:

```
404 Not Found
```

Now we need to ask:

```
Did Spring detect my controller?  
Is the package structure correct?  
Did component scanning happen?  
Was the bean created?  
Did Spring MVC register the mapping?  
Is the URL correct?  
Did Tomcat receive the request?
```

These questions cannot be answered by memorizing annotations.

They require understanding the internal flow.

Example: Application Starts Slowly

Suppose the application takes 10 seconds to start.

Spring Boot will start the application, but we still need to understand what may be happening:

```
Too many beans being created  
Database connection taking time
```

```
Auto-configuration doing extra work  
Heavy startup logic running  
External service call blocking startup
```

To debug this, we need to understand Spring Boot internals.

Example: Controller Does Not Receive R

Suppose the browser sends a request, but the controller never receives it.

Possible reasons could be:

```
Wrong port  
Wrong URL  
Application not running  
Controller not detected  
Incorrect annotation  
Context path configured  
Request blocked by security
```

Again, solving this requires understanding the layers behind Spring Boot.

21. The Industry Reality

In real projects, companies do not only pay engineers for writing:

```
@RestController
```

That part is easy.

AI tools, code generators, and IDEs can write basic code.

Companies pay engineers for understanding:

```
Why something is not working  
How different layers communicate  
How to debug production issues  
How to design maintainable systems  
How to make the application reliable
```

That is why learning the internals is important.

Spring Boot gives speed.

Spring fundamentals give control.

22. Final Summary

In this lecture, we built our first Spring Boot web application.

We learned:

```
A browser sends a request to a server.  
A port identifies the application running inside a machine.  
localhost means our own machine.  
localhost:8080 means our own machine on port 8080.  
Spring Initializr helps us generate a Spring Boot project quickly.  
A dependency is an external library required by our project.  
@RestController marks a class as capable of handling web requests.  
@GetMapping("/hello") maps a GET request to a Java method.  
Spring Boot starts and configures the application.  
Embedded Tomcat listens for HTTP requests.  
The browser receives the response and displays it.
```

The most important thing to remember:

Today we built a web application very quickly, but many powerful concepts were working behind the scenes.

In the upcoming lectures, we will slowly understand those concepts one by one:

```
Maven  
Servlets  
Tomcat  
Spring Core  
IoC  
Dependency Injection  
Beans  
Spring MVC  
Spring Boot auto-configuration
```

Once these foundations are clear, Spring Boot will no longer feel magical.

It will start making logical sense.